



## Web Technologies – SEF23

### Homework 07- Travel Booking System (with Identity)

18<sup>th</sup> Nov, 2025 – Professor Shuja Ur Rehman Baig

#### Important Instructions:

- Any instance of cheating or plagiarism will result in a **F grade**.
- Deadline: 18<sup>th</sup> Nov, 2025 11:59 PM

---

This homework extends the previously developed **Travel Booking System (HW5)** by incorporating authentication and authorization using ASP.NET Core Identity. The goal is to implement user roles (Admin and User) and role-based dashboards, allowing secure access and controlled functionality.

#### Learning Objectives

- Implement Authentication and Authorization using ASP.NET Core Identity
- Configure Role-based Access (Admin and User)
- Secure Controllers and Views with Authorization Attributes
- Integrate Identity with existing SQL Server database
- Build role-specific dashboards and functionalities

#### Part 1: Identity Integration Setup

1. Integrate ASP.NET Core Identity into the existing Travel Booking System project.
2. Update the database to include Identity tables (AspNetUsers, AspNetRoles, etc.).
3. Configure Identity in Program.cs using `services.AddIdentity<IdentityUser, IdentityRole>()`.
4. Enable authentication middleware using `app.UseAuthentication()` and `app.UseAuthorization()`.

#### Part 2: Roles Setup

1. Create two roles: Admin and User.
2. Automatically seed these roles when the application starts. Admin account must be there by default.
3. When a new user signs up, assign them the 'User' role by default.



## Part 3: Authentication Pages

1. Create Sign Up and Login pages using Razor Views.
2. Use Identity's built-in UserManager and SignInManager for user management.
3. After successful login:
  - Users are redirected to the User Dashboard.
  - Admins are redirected to the Admin Dashboard.

## Part 4: User Dashboard

1. Accessible only to users with the 'User' role.
2. Displays available travel packages (reusing previous functionalities).
3. Allows users to make bookings as before.
4. Bookings made by users will be marked as 'Pending' by default.

## Part 5: Admin Dashboard

1. Accessible only to users with the 'Admin' role.
2. Displays a list of all bookings made by users.
3. Admin can approve or reject each booking.
4. Approved or Rejected status should update in the database and reflect in the user's dashboard.

## Part 6: Database Modifications

1. Modify the Bookings table to include a new column:  
`BookingStatus NVARCHAR(20) DEFAULT 'Pending'`
2. Update queries and models accordingly to handle the new column.

## Part 7: Authorization Rules

- Decorate controllers with [Authorize] attribute.
- Restrict access to Admin Dashboard with [Authorize(Roles = "Admin")].
- Restrict access to User Dashboard with [Authorize(Roles = "User")].